COMP2511

Tute07

Agenda

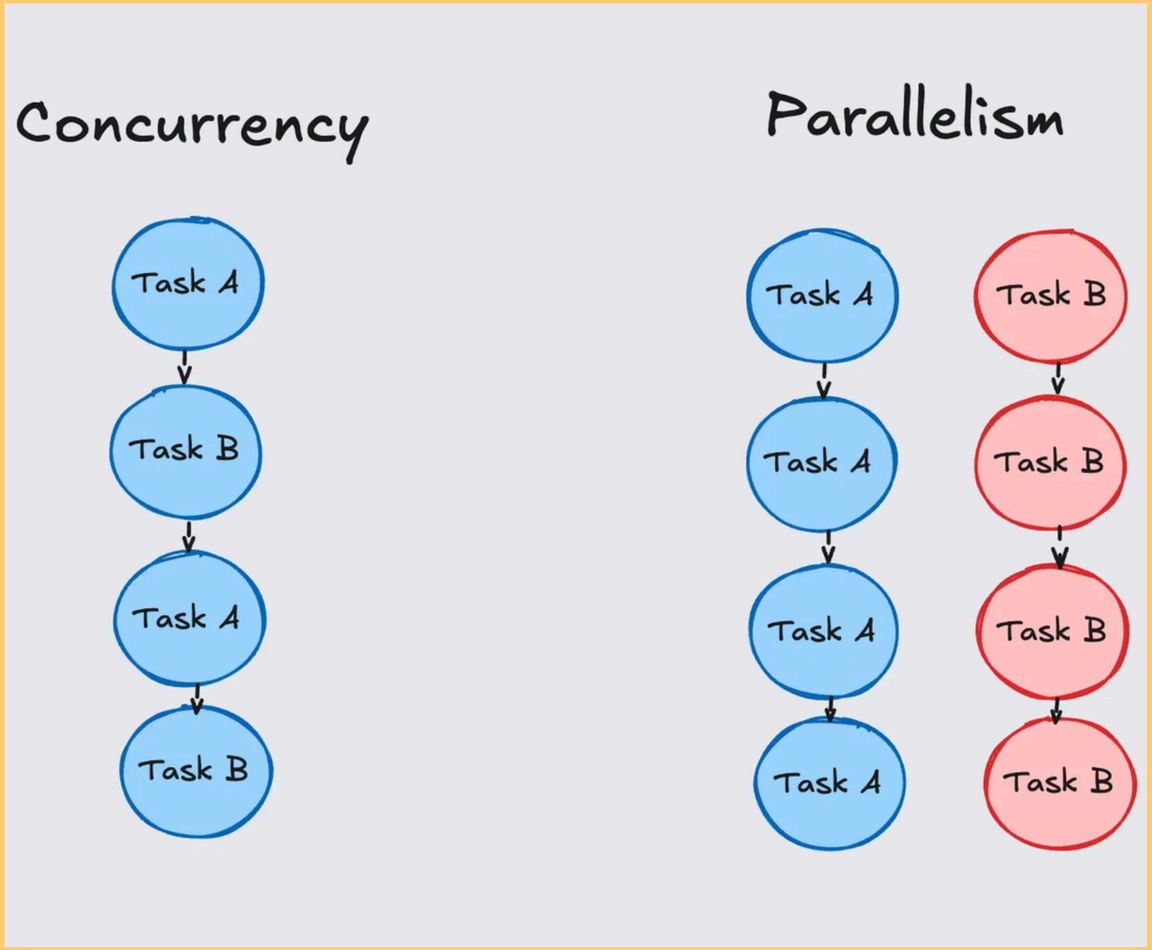
- Concurrency, Multithreading & Data Races
- Singleton Pattern
- Abstract Factory Pattern
- Decorator Pattern





Concurrency?

What is concurrency?



Concurrency

Concurrency is the ability of a program to execute different sections of code seemingly simultaneously. We achieve this through

Order Of Operations

SINGLE THREADING

Masterchef cooking one dish at a time.

MULTI THREADING

Sous chefs cooking every dish simultaneously.

ASYNCHRONOUS

Masterchef directing sous chefs on what to do.

Singleton Pattern





Singleton Pattern

The common problem?

Some part of the code is very sensitive to concurrency (deadlocks, race conditions)

- A log file is being accessed by many processes simultaneously
- A JSON file is being read and written into by concurrent processes



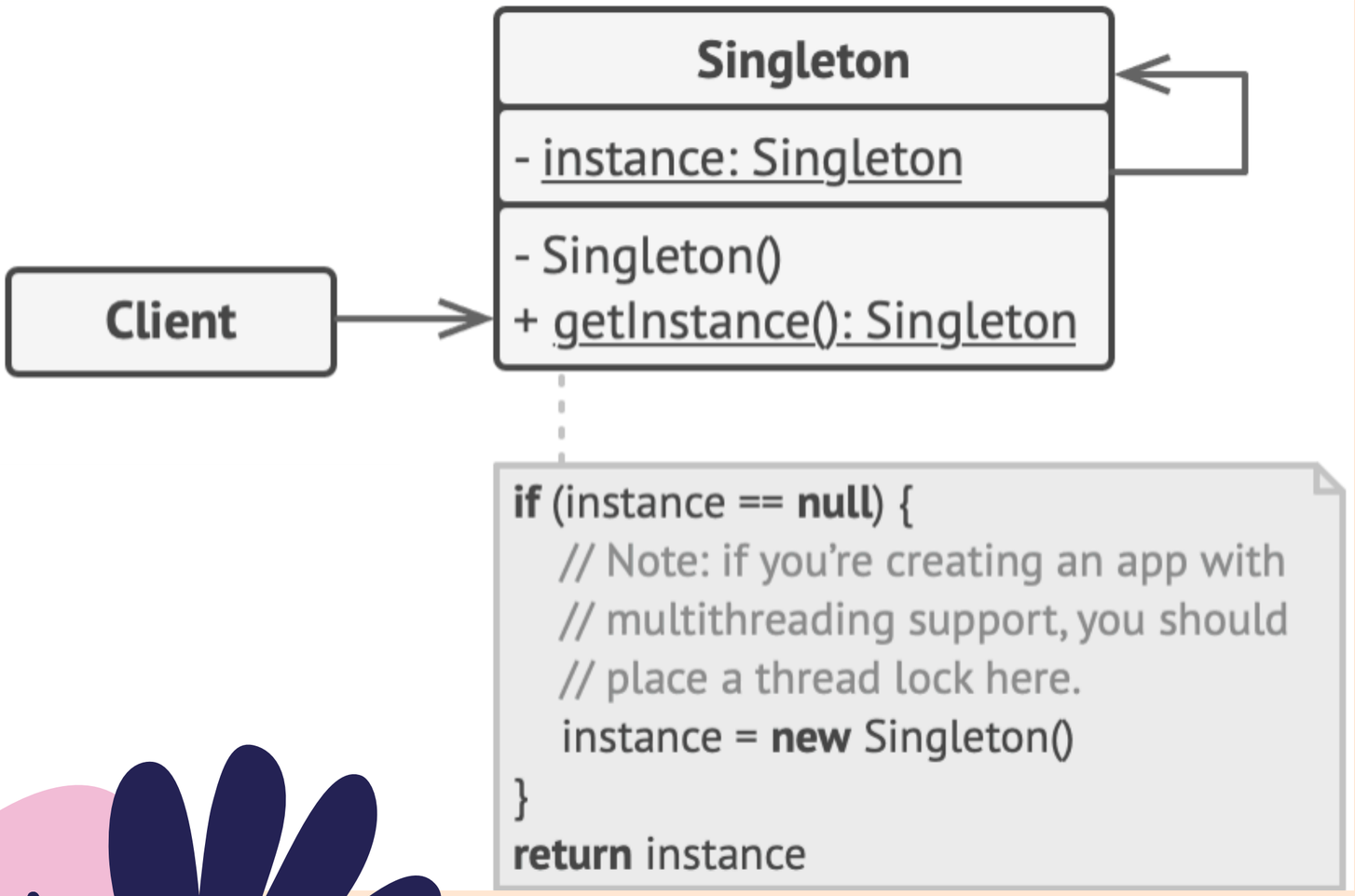
Singleton Pattern

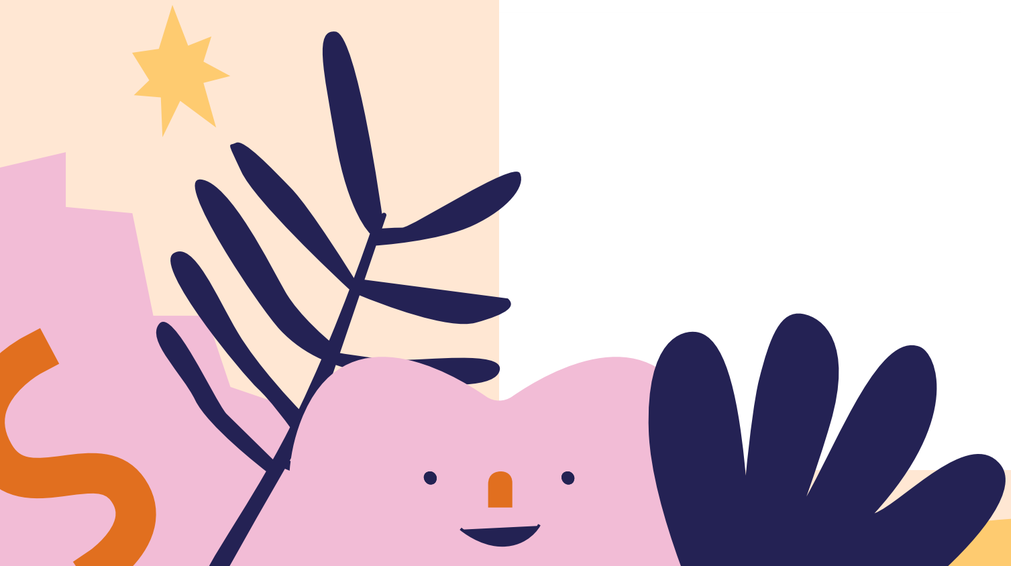
The common solution?

1. Identify and encapsulate concurrency-prone objects into classes.
2. Ensure that the class has a single instance.
3. Provide a global access point to that single instance.



Singleton Pattern





Overcooked





Abstract Factory Pattern





Abstract Factory

The common problem?

In our code, we need to create entire families of related objects without specifying their concrete classes.

- Producing beds, chairs, and tables for a Victorian/Modern/WWII household.
- Spawning grass + oak for Plain or sand + cactus for Desert or snow + spruce for Snowy Taiga



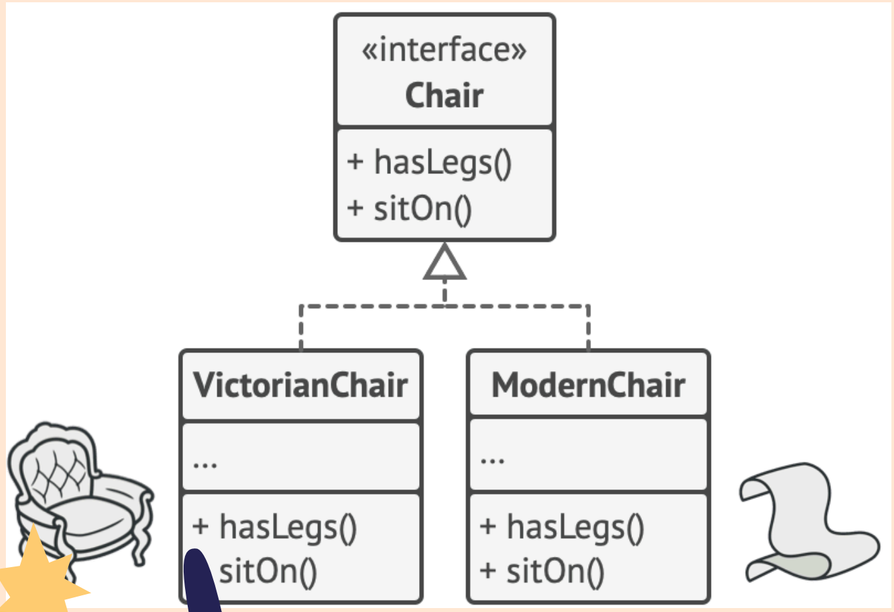
Abstract Factory

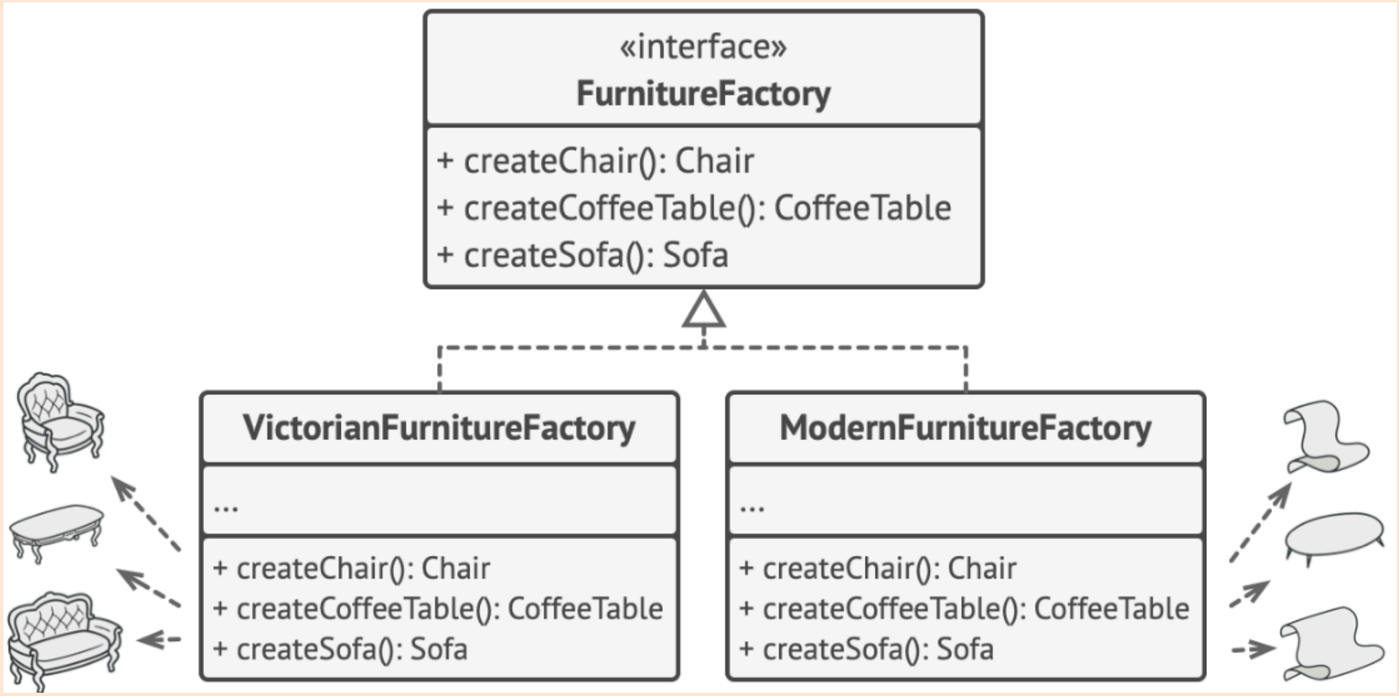
The common solution?

1. Identify the products (ie. beds chairs tables or blocks fauna mobs).
2. Identify the variations (ie. Victorian Modern WWII or biomes).
3. Create an interface for each product.
4. Create an Abstract Factory (interface).
5. Create a concrete factory for each variation implementing the abstract factory.



Abstract Factory







Abstract?

Use abstract for repetitive systems where objects have variations. Stick with factory for all else.

Thrones





Decorator Pattern



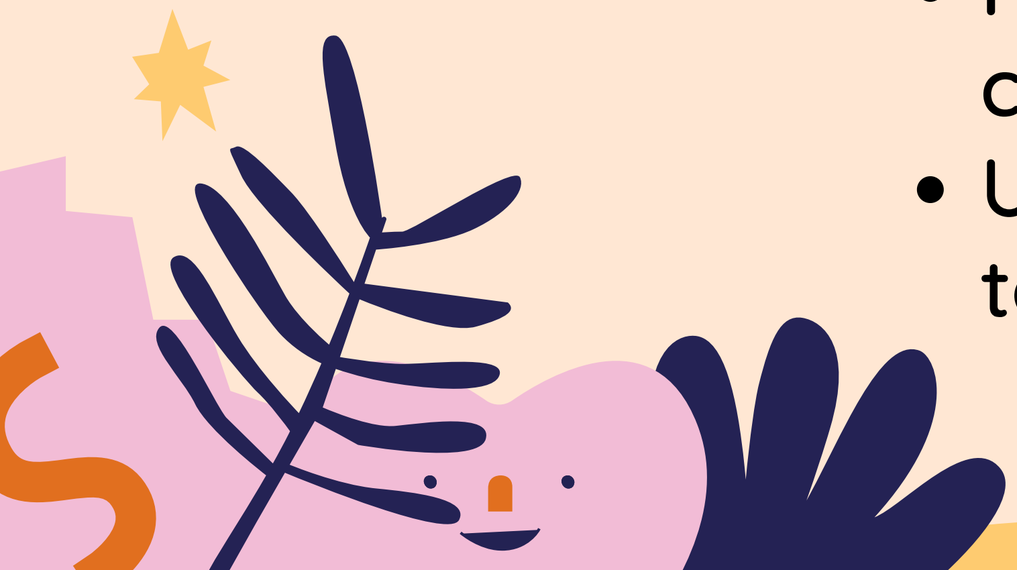


Decorator Pattern

The common problem?

In our code, we can continuously add stuff to an object that changes it.

- Yoichi has unlimited topping combinations.
- Humans can wear any number of clothes and accessories.
- Unlimited decorations can be added to a present.



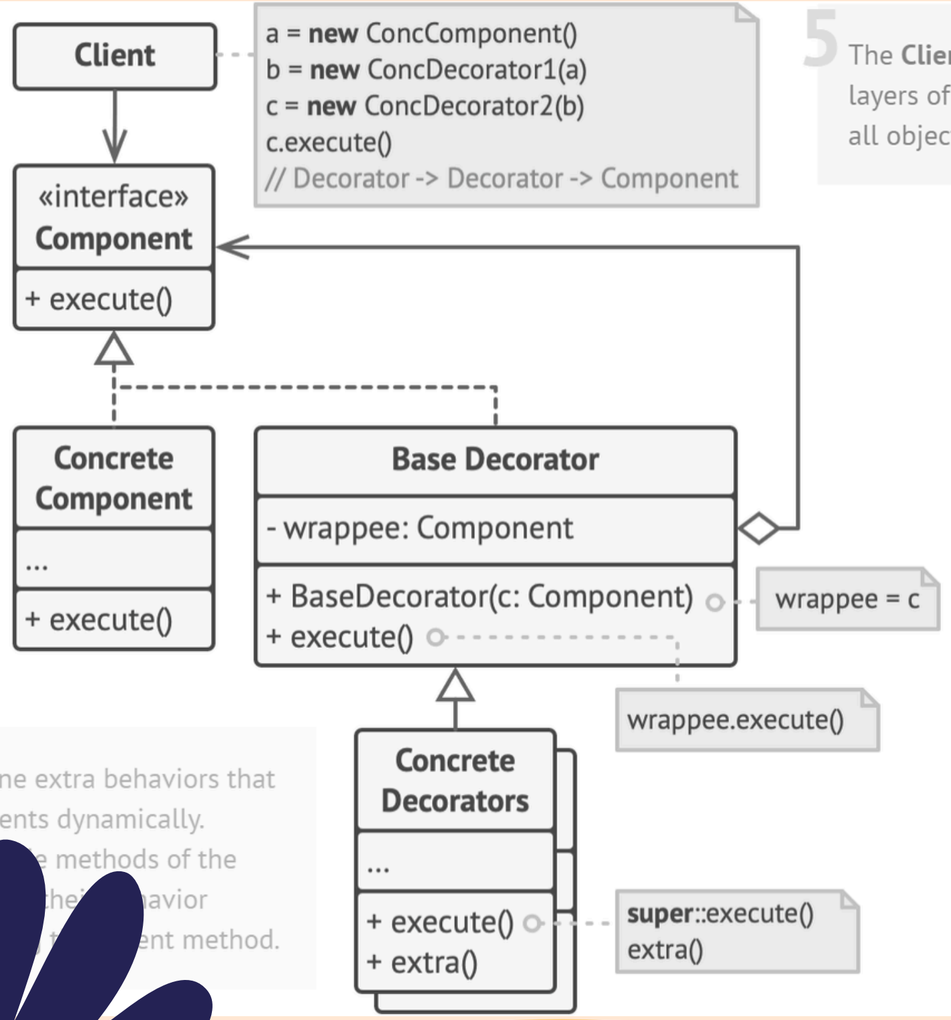
Decorator Pattern

The common solution?

1. Identify the object being decorated.
2. Identify the decorations.
3. Add an interface for the decorated object.
4. Create a base decorator class that contains an instance of the interface.
5. Extend the base decorator to create a concrete decorator for each decoration.



Decorator Pattern



1 The **Component** declares the common interface for both wrappers and wrapped objects.

2 **Concrete Component** is a class of objects being wrapped. It defines the basic behavior, which can be altered by decorators.

3 The **Base Decorator** class has a field for referencing a wrapped object. The field's type should be declared as the component interface so it can contain both concrete components and decorators. The base decorator delegates all operations to the wrapped object.

4 **Concrete Decorators** define extra behaviors that can be added to components dynamically. Concrete decorators ov e methods of the base ex the avior ent method.

5 The **Client** can wrap components in multiple layers of decorators, as long as it works with all objects via the component interface.

Thrones part ii



